

Physarum - a walkthrough

Physarum

Three hundred thousand agents, each obeying one rule that fits in a sentence, and a branching vein network that no line of code describes.

An agent looks three ways. It turns toward whichever direction smells strongest. It steps forward, and leaves a trace. That is the whole behaviour — and no agent knows about any other agent, and no cell of the trail map knows it is part of a network.

The branching, converging veins that a real slime mould grows to solve a maze come out of that alone.

The shortest possible summary

There are **two** populations on the GPU, and that is what makes this example different from every other one in the collection. The agent kernel is addressed by *agent index* — 300,000 threads, one per creature. The trail kernels are addressed by *pixel index* — 589,824 threads, one per cell. They meet only through a shared map that both read and write.

That is **stigmergy**: coordination achieved by modifying a shared environment, rather than by any agent communicating with any other. Termites build mounds this way. Ants find food this way. Here, three hundred thousand independent walkers grow a transport network without any of them being able to see it.

The trail map gets its own treatment once a frame — a nine-point box blur that spreads it, times a decay factor that fades it. Without spreading, an agent could never sense a neighbour's trail. Without fading, every path ever taken would persist and the map would saturate to uniform bright.

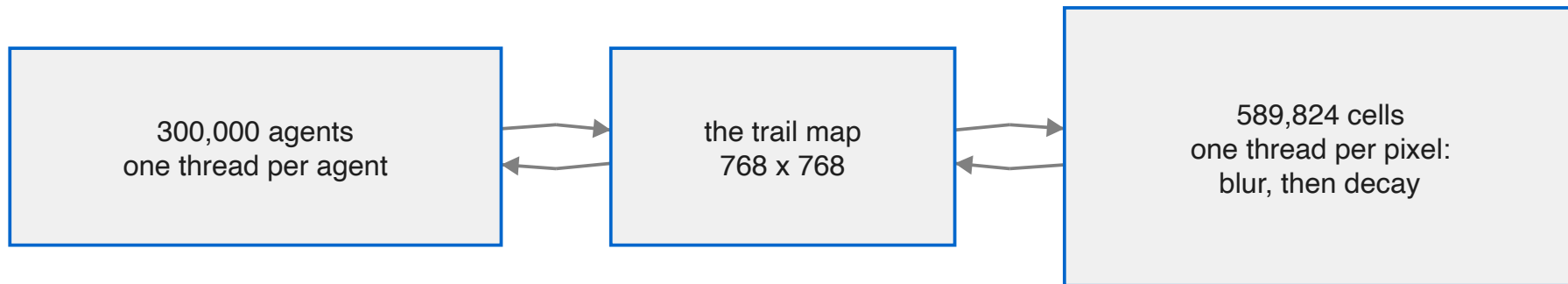
A trail is a rumour: it spreads to its neighbours and fades unless agents keep repeating it.

These documents

Chapter	What it covers
1. The organism	A mould that solves mazes, and the model that reduced it to one rule
2. The rule	Sense, turn, step, deposit — and why the trail has to fade
3. Why this belongs on a GPU	Two populations at once, and the race the file accepts on purpose
4. The five kernels	Every kernel line by line, and hashing instead of an RNG
5. A frame, end to end	Three dispatches, one ping-pong, and the parity trick
6. What to understand	The things that will surprise you, and the ones that will bite

At a glance

Source	examples/physarum/main.mojo, 738 lines
Agents	300,000 — one thread each
Trail map	768 × 768 — 589,824 cells, one thread each
Per frame	3 dispatches: agents, diffuse, colour



1. The organism

A single cell that solves mazes

Physarum polycephalum is a slime mould: not a plant, not a fungus, not an animal, and — for the part that matters here — **not made of cells**. In its plasmodial stage it is one enormous cell with many nuclei, a spreading yellow sheet that creeps over rotting wood looking for food.

It has no brain. It has no nervous system. It has nothing that could be called a component that makes decisions.

In 2000, Toshiyuki Nakagaki and colleagues published a short paper in *Nature* titled "**Maze-solving by an amoeboid organism**". They grew a plasmodium across an agar maze, placed oat flakes at the entrance and the exit, and came back. The mould had withdrawn from every dead end and left a single thick vein along the **shortest path** between the two food sources.

Ten years later the same group did something harder. In "**Rules for Biologically Inspired Adaptive Network Design**" (*Science*, 2010), Atsushi Tero and colleagues laid oat flakes on a map of the Tokyo area at the positions of the major stations and let a plasmodium grow across it. The network it produced was comparable to the actual Tokyo rail network in total length, transport efficiency and tolerance to a severed link.

Both results won Ig Nobel Prizes, and both are the reason anyone in computing cared: **a network with no designer, arriving at the answer a designer arrived at.**

The model: one rule instead of an organism

What this example runs is not a simulation of slime-mould biology. It is a much later, much simpler thing.

Jeff Jones's 2010 work on approximating Physarum transport networks reduced the behaviour to a population of identical agents, each carrying only a position and a heading, each doing four things per step:

1. **sense** the trail map at three points ahead — left, centre, right
2. **turn** toward whichever reads strongest
3. **step** forward
4. **deposit** into the map at its new position

Plus one rule for the map itself: **spread and fade**.

That is the entire model. It is not derived from the organism's chemistry; it is an observation that agents which follow their own accumulated traces produce networks with the same character. The example's header states the claim exactly at that level:

the branching, converging veins that a real slime mould grows to solve a maze emerge from that alone, the same way the reef in grayscott emerges from two numbers rather than being drawn.

Stigmergy

The mechanism has a name, and it is older than the slime-mould work.

Pierre-Paul Grassé, studying termite mound construction in 1959, needed a word for the fact that termites were plainly cooperating while plainly not communicating. His answer was that they were not coordinating with each other at all: each termite responded to **the state of the structure**, and its response changed that state, which changed what the next termite responded to. He called it **stigmergy** — coordination through the environment.

Ant pheromone trails are the same idea. So is this program:

No agent knows about any other agent, and no cell of the map knows it is part of a network.

There is no communication in the file. There is no agent-to-agent lookup, no neighbour list, no spatial index. There is a shared array that everyone reads and everyone writes, and that is sufficient.

Why the fading matters

The decay term is not an aesthetic choice, and it is what makes stigmergy work rather than merely accumulate.

Without decay, every path ever taken persists. After a few hundred frames the map is uniformly bright, all three sensors read the same value, and the agents turn randomly forever — a network that has dissolved into noise by remembering too much.

Decay makes the trail a **claim that has to be renewed**. A route stays visible only while agents keep using it; a route nobody takes disappears. That is precisely the dead-end withdrawal Nakagaki photographed, and it falls out of a multiply.

```
out_t[unsafe_offset=idx] = (s / Float32(9.0)) * decay
```

The source puts it better than a paragraph can:

A trail is a rumour: it spreads to its neighbours and fades unless agents keep repeating it.

Four presets, and an honest note about them

```
# Unlike Gray-Scott's feed/kill plane, Physarum has no narrow sweet spot --  
# nearly any sensor angle, turn angle and sensor distance produce SOME  
# branching network, so these four are this file's own tuning rather than  
# values looked up from a paper.
```

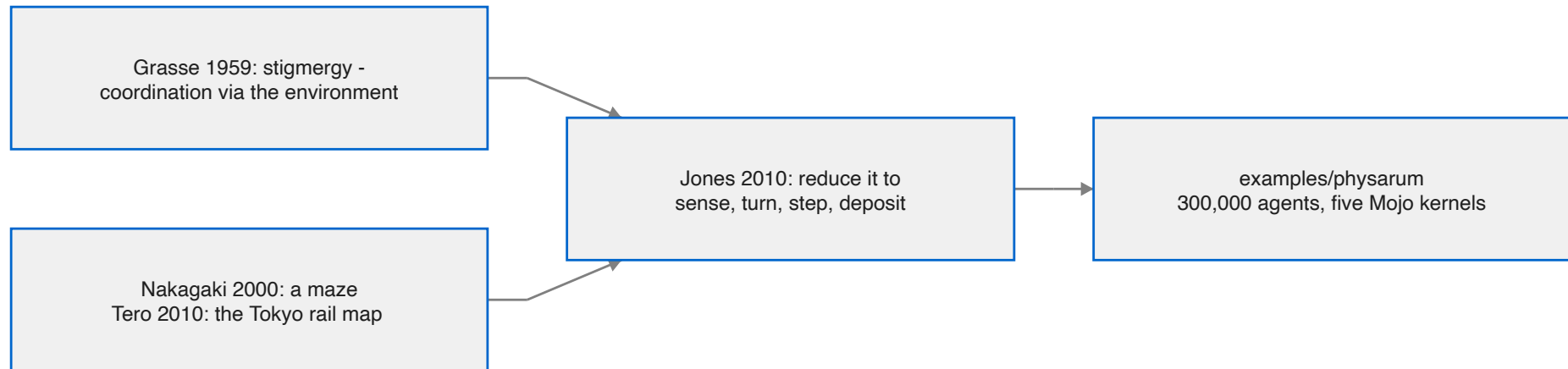
preset	sensor angle	turn angle	sensor distance	decay
web	0.45	0.30	9.0	0.94
coil	0.20	0.50	6.0	0.92
sparse	0.60	0.25	16.0	0.96
dense	0.35	0.35	5.0	0.90

That comment is worth reading beside [Gray-Scott's](#), which says the opposite about its own numbers — that they are *"the widely reproduced values this model is usually shown with, not a guess"*, because being a few thousandths out gives a uniform grey.

Two examples in the same collection, two different provenances, each labelled. Gray-Scott's parameter plane has sharp boundaries and a published map, so its values are cited. Physarum's does not, so its values are described as tuning and not dressed up as anything else.

The presets read as a coherent set once you know what each dial does:

- **coil** — sensors close together (0.20) and sharp turns (0.50): an agent commits hard to small differences, so trails curl.
- **sparse** — sensors wide (0.60) and far ahead (16.0), turns gentle (0.25), decay slow (0.96): agents commit to long straight runs and the traces persist, giving long branches.
- **dense** — everything short and fast-fading: many small features.



2. The rule

An agent is three floats: `x`, `y`, and a heading `a`. Not a struct — three separate arrays, which [chapter 3](#) explains.

Everything an agent ever does is in one kernel, and the docstring is the specification:

Sense three points ahead, turn toward the strongest, step, deposit. That is the entire behaviour — an agent holds no memory of where it has been, and reacts to nothing but the shared map every other agent is also reading and writing.

Sense

```
var sl = _at(trail, Int(x + cos(a - sensor_angle) * sensor_dist),  
             Int(y + sin(a - sensor_angle) * sensor_dist))  
var sc = _at(trail, Int(x + cos(a) * sensor_dist),  
             Int(y + sin(a) * sensor_dist))  
var sr = _at(trail, Int(x + cos(a + sensor_angle) * sensor_dist),  
             Int(y + sin(a + sensor_angle) * sensor_dist))
```

Three samples: `sensor_angle` to the left of the heading, straight ahead, and the same angle right. Each at `sensor_dist` away.

Two parameters, and between them they set the agent's whole character:

- **sensor_angle** is how *different* the three directions are. Narrow (coil, 0.20) and the agent is choosing between three nearly-identical options, so tiny differences in the map decide its turn. Wide (sparse, 0.60) and it is comparing genuinely different directions.
- **sensor_dist** is how far ahead it looks. Short (dense, 5.0) and it responds to what is immediately in front — fine, twitchy structure. Long (sparse, 16.0) and it commits to distant features, ignoring what it is walking over.

Note there is no gradient, no derivative, no least-squares fit to the field. Three samples and two comparisons.

Turn

```
if sc > sl and sc > sr:
    pass
elif sl > sr:
    a -= turn_angle
elif sr > sl:
    a += turn_angle
else:
    var r = _rand01(UInt32(idx) * UInt32(747796405) + frame)
    a += turn_angle if r > Float32(0.5) else -turn_angle
```

Straight ahead wins ties against the sides — `sc > sl and sc > sr` first — so an agent on a good trail keeps going rather than wobbling. Otherwise it turns `turn_angle` toward the better side.

Note the turn is a **fixed step, not proportional to the difference**. An agent sensing a marginally brighter left turns exactly as hard as one sensing a vastly brighter left. That is a modelling choice with a visible consequence: agents cannot ease into a curve, so trails are made of small discrete corrections, and the characteristic filament texture comes from that.

The `else` branch is the interesting one. If left and right read *exactly* equal — which on an empty map is every agent on the first frame, and later happens wherever the field is flat — the agent picks a side at random.

Without it, a tie would leave the heading unchanged and every agent would walk in a straight line until it happened to find a trail. The map would stay blank far longer, and the initial symmetry would break much more slowly.

And the randomness is a **hash**, not a generator — see [chapter 4](#).

Step

```
var nx = x + cos(a) * Float32(1.2)
var ny = y + sin(a) * Float32(1.2)
```

```
if nx < Float32(0.0):
    nx += Float32(WIDTH)
elif nx >= Float32(WIDTH):
    nx -= Float32(WIDTH)
```

1.2 pixels per step, and the world **wraps**. An agent leaving the right edge reappears on the left.

That decision propagates, and the source is explicit about the consequence:

```
def _at(f, x, y) -> Float32:
    """The trail map wraps, not clamps: an agent that walks off the right
    edge reappears on the left, so the map it senses and deposits into
    must wrap the same way or a trail would smear against a false wall
    the agents themselves never see."""
```

This is the one place Physarum departs from Gray-Scott's discipline, and for a good reason. Gray-Scott **clamps** — its cells do not move, so a reflecting boundary is the natural physical choice. Physarum's agents *travel*, so a clamped map would give them a torus to walk on and a rectangle to sense on: an agent that wrapped from $x = 767$ to $x = 0$ would find its own trail apparently piled up against a wall that only the map believes in.

A 1.2-pixel step is also deliberate. Less than one and agents deposit into the same cell repeatedly, making dotted trails; much more and they skip cells, so a trail becomes a dashed line the sensors read as gaps.

Deposit

```
var cidx = _wrap(Int(ny), HEIGHT) * WIDTH + _wrap(Int(nx), WIDTH)
trail[unsafe_offset=cidx] = trail[unsafe_offset=cidx] + DEPOSIT
```

Add `DEPOSIT` (5.0) to the cell the agent has landed on. Read, add, write — and **not atomically**, which is a deliberate decision covered in [chapter 3](#).

Note that the agent deposits at its **new** position, having already turned and moved. So the trail records where agents *went*, not where they were.

And the map's own rule

Agents alone are not enough. A trail deposited into one cell is invisible to every agent that is not aimed exactly at that cell — so the map has to spread what it receives:

```
"""A trail is a rumour: it spreads to its neighbours and fades unless
agents keep repeating it. The nine-point box average IS the spreading;
multiplying by `decay` afterward is the fading. Both happen to every
cell whether or not an agent ever visited it -- there is no separate
'has this pixel ever seen an agent' bit anywhere."""
```

```
s += _at(in_t, x - 1, y - 1)
... nine of them ...
out_t[unsafe_offset=idx] = (s / Float32(9.0)) * decay
```

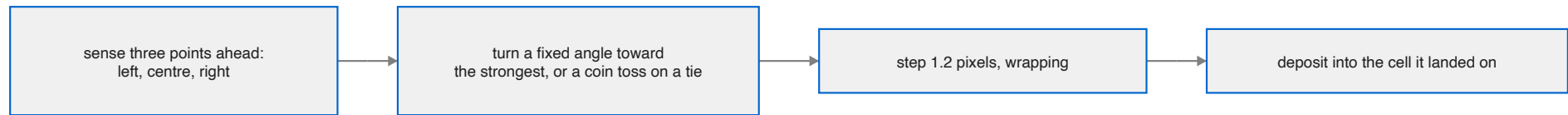
A plain nine-point **box average** — all weights $1/9$ — and then a multiply.

Worth comparing with Gray-Scott's Laplacian, which looks similar and is not:

	Physarum	Gray-Scott
weights	all $1/9$	$0.05 / 0.2 / -1.0$
they sum to	one	zero
what it returns	a value — the local average	a rate of change
what it is	a blur	a Laplacian

Both are nine-point stencils over the same neighbourhood, and they do opposite things. Gray-Scott needs a rate of change because it *integrates* it into an existing field. Physarum wants the average itself, because the average **is** the new value.

Blur and decay are one kernel and one multiply, and between them they are the entire memory of the system.



3. Why this belongs on a GPU

Physarum is a good GPU workload for different reasons than [Gray-Scott](#), and the differences are the interesting part.

Two populations, which is the new shape

Every other GPU example in this collection dispatches over **one** thing. Mandelbrot: pixels. Fluid: grid cells. Gray-Scott: grid cells. Fernwind: chaos-game streams.

Physarum has two, and the commit names it as the point of the example:

one genuinely new shape: two different populations on the GPU at once, an agent kernel addressed by agent index and a trail kernel addressed by pixel index, rather than grayscott's one grid doing everything.

```
comptime GRID = (CELLS + BLOCK - 1) // BLOCK      # 2,304 blocks
comptime AGENT_GRID = (AGENTS + BLOCK - 1) // BLOCK # 1,172 blocks
```

Two grid sizes, and each kernel is launched with the one matching what it iterates over. The agent kernel does not know the map is 768 × 768 except as a wrap modulus; the diffuse kernel does not know agents exist at all.

They share exactly one thing: a `Pointer[Float32]` to the trail map.

Agents are a structure of arrays

```
def agent_kernel(px: Pointer[Float32, MutAnyOrigin],
                 py: Pointer[Float32, MutAnyOrigin],
                 pa: Pointer[Float32, MutAnyOrigin], ...)
```

Not an array of `Agent {x, y, angle}` structs. Three separate arrays.

That is **structure-of-arrays** layout, and on a GPU it is not a style preference. Thread n reads `px[n]`, thread $n+1$ reads `px[n+1]` — adjacent threads touch adjacent addresses, so the hardware coalesces 32 loads into one transaction. With an array of structs, thread n would read offset $12n$ and consecutive threads would stride past two floats they do not want, turning one transaction into several.

Three arrays, three coalesced streams, for a layout that reads slightly worse in source and runs meaningfully better.

What each half looks like to the hardware

The trail kernels: textbook

`diffuse_kernel` is a nine-point stencil over 589,824 cells with a fixed neighbourhood, uniform control flow, no reduction and no atomics. Everything Gray-Scott's chapter 3 says applies unchanged.

`color_kernel` and `attract_kernel` are the same shape, one thread per pixel.

The agent kernel: mostly good, honestly

Per agent: three `_at` reads, six trig calls, a handful of comparisons, three writes to its own slots, one read-modify-write into the map.

Two properties are excellent:

- **Perfectly independent.** No agent reads another agent's position. There is no neighbour search and no spatial data structure — the map does the coupling.
- **No dynamic allocation, no variable work.** Every agent does exactly the same amount of work every step. Nothing is born, nothing dies, nothing has a list.

And two are worth being honest about:

- **The turn is a branch.** `if sc > sl and sc > sr ... elif ... else` is data-dependent, and adjacent agents will take different arms. It is short — the arms differ by one addition — so the hardware masks rather than serialising anything expensive, but it is not free the way Gray-Scott's straight-line arithmetic is.

- **The trail reads are scattered.** Agents at adjacent indices are at unrelated positions after a few hundred frames, so their sensor reads land anywhere in the map. This is the opposite of the stencil kernels' neat locality, and it is the agent kernel's real cost. Nothing can be done about it without sorting agents by position, which would cost more than it saves at this scale.

The race the file accepts on purpose

```
trail[unsafe_offset=cidx] = trail[unsafe_offset=cidx] + DEPOSIT
```

Read, add, write — from 300,000 threads into one shared array, **without an atomic**. Two agents landing on the same cell in the same step can lose an increment.

The kernel says so itself, in its own docstring:

The deposit at the end is a plain, non-atomic add: two agents landing on the same cell in the same step can lose one increment to the other, and at 300,000 agents over 589,824 cells that collision is rare and, once it happens, invisible — a fraction of one step's trail on one pixel, immediately blurred and decayed with everything around it. An atomic add would buy correctness this demo has no way to show.

Three separate claims, and each is checkable.

Rare. 300,000 agents over 589,824 cells is roughly one agent per two cells. If they were uniformly scattered, collisions would be a small fraction of deposits. They are *not* uniformly scattered — agents concentrate on trails, which is the whole point — so the real rate is higher on a busy trunk route than the average suggests.

Invisible. A lost deposit is 5.0 on one cell for one frame. That cell is then averaged with its eight neighbours and multiplied by ~0.94, and it is one of 589,824. There is no display path on which the difference could appear.

An atomic would buy correctness the demo cannot show. This is the honest version of the argument, and the one worth keeping. It is not "atomics are slow" — it is that the *only* consequence of the race is a value nobody can distinguish from the value without it.

Compare [fernwind](#), which does the same kind of scattered accumulation and **does** use `Atomic.fetch_add` — because there the accumulated count is divided to produce a mean colour, so lost increments would bias the result systematically rather than dither it.

Same operation, opposite decisions, both reasoned from what the number is afterwards used for. That is the transferable part: *"is this racy?"* is the wrong question, and *"what would a lost increment change?"* is the right one.

No RNG state anywhere

A conventional agent simulation gives each agent a random-number generator and advances it. That is per-agent state to allocate, initialise and carry.

This one hashes instead:

```
# GPU threads have no shared RNG state to advance -- each one HASHES its own
# index (and the frame count, when the same agent needs a fresh number every
# step) instead, which is embarrassingly parallel by construction and needs
# no setup kernel of its own.
```

```
var r = _rand01(UInt32(idx) * UInt32(747796405) + frame)
```

Index and frame in, a number out, no state and no ordering. The initial scatter uses the same trick with three different multipliers, so 300,000 agents get random positions and headings **on the GPU** with no host-side random calls at all.

The cost, and the comparison

Three dispatches per frame — agent, diffuse, colour. That is the cheapest frame of any simulation in the collection.

example	dispatches per frame	what couples the work
mandelbrot	1	nothing
physarum	3	a shared map, written racily
grayscott	21	one cell of diffusion per substep
fluid	~35	a global constraint, needing a solver

Physarum is cheap because its coupling is *weak*: agents affect each other only through a map that is blurred and decayed every frame anyway. Fluid is expensive because its coupling is *exact* — incompressibility has to hold everywhere, at once, to a tolerance.

Reading those four together is the clearest picture in the collection of what coupling costs.

4. The five kernels

Five kernels, all ordinary Mojo `def`s through this fork's AIR backend. Two are addressed by agent index, three by pixel index.

kernel	threads	when
init_agents	one per agent	once, and on r
agent	one per agent	every frame
diffuse	one per cell	every frame
attract	one per cell	only while dragging
color	one per cell	every frame

The hash that replaces a random generator

```
def _hash_u32(x: UInt32) -> UInt32:
    var h = x
    h = (h ^ UInt32(61)) ^ (h >> UInt32(16))
    h = h + (h << UInt32(3))
    h = h ^ (h >> UInt32(4))
    h = h * UInt32(0x27D4EB2D)
    h = h ^ (h >> UInt32(15))
    return h
```

Six operations, no memory, no state. Shifts and xors mix the low bits upward and the high bits downward; the odd multiplier spreads everything; the final shift-xor finishes the avalanche.

```
def _rand01(seed: UInt32) -> Float32:
```

```
return Float32(_hash_u32(seed) & UInt32(0xFFFFF)) / Float32(0xFFFFF)
```

Twenty-four bits, which is exactly what a `Float32` mantissa holds without rounding. Taking all 32 would divide by a number the float cannot represent and quietly lose the low bits.

The reason it is a hash rather than a generator is in the comment: a generator has **state**, and state has to be allocated, seeded and carried between calls. 300,000 of them would need their own buffer and their own setup kernel. A hash of the index needs nothing.

1. init_agents_kernel

```
var rx = _rand01(UInt32(idx) * UInt32(2654435761) + UInt32(1))
var ry = _rand01(UInt32(idx) * UInt32(2246822519) + UInt32(2))
var ra = _rand01(UInt32(idx) * UInt32(3266489917) + UInt32(3))
```

entirely on the GPU, hashing each agent's own index rather than needing 300,000 separate host-side random calls before the first frame can run

Three different large odd multipliers and three different offsets, so the same `idx` produces three uncorrelated values. Using the same seed three times would put every agent on the diagonal with a heading proportional to its position, and the first frame would look like a diffraction grating.

2. agent_kernel — the whole behaviour

Covered in [chapter 2](#). Two things about its *shape*:

Parameters are arguments, not constants. `sensor_angle`, `turn_angle`, `sensor_dist` arrive per dispatch, so the number keys switch regime live without recompiling — the same choice Gray-Scott makes with `feed` and `kill`.

frame is an argument too, and it is what makes the tie-breaking random number differ from step to step for the same agent. Without it, an agent that found a tie would break it the same way every frame forever.

3. `diffuse_kernel` — spread and fade

```
out_t[unsafe_offset=idx] = (s / Float32(9.0)) * decay
```

A nine-point box average and a multiply. Separate in and out buffers, for the same reason every stencil in this tree has them: a cell's new value depends on its neighbours' **old** values, and writing in place would let a thread read neighbours another thread has already updated.

`_at` wraps here rather than clamping, matching the agents' own topology — [chapter 2](#) covers why that has to be consistent.

4. `attract_kernel` — the mouse

```
if dx * dx + dy * dy <= r2:  
    trail[unsafe_offset=idx] = Float32(400.0)
```

Sets a disc of the trail map to a large value — note **set**, not add, so holding the mouse still does not accumulate without limit.

The docstring is the best sentence in the file:

A strong trail laid down under the cursor. Nothing forces an agent toward it — the network only bends here because sensing a bright patch and turning toward it is all an agent ever does anyway; this just gives it something bright to find.

There is no attraction code. There is no force, no target, no steering behaviour. The mouse writes into the same map the agents were already reading, and the network bends because of a rule that was there before the mouse existed.

That is what a well-factored emergent system looks like: the interaction is free because it uses the mechanism that was already the whole model.

5. color_kernel — tone mapping, borrowed

```
var m = t / (Float32(1.0) + t)
var r = m
var g = m * m * Float32(0.85)
var b = m * m * m * Float32(0.55)
```

fluid's own $x/(1+x)$ curve, reused for the same reason it was needed there: raw deposit totals span a huge range between a cell one agent grazed once and a cell on a busy trunk route, and a linear map would blow the second one out to solid white while the first stayed too dark to see.

$x/(1+x)$ maps $[0, \infty)$ into $[0, 1)$ — a cell with 5.0 comes out at 0.83, one with 400 at 0.9975, and nothing ever reaches 1.0.

The colour ramp is three different **powers of the same value**: red linear, green squared, blue cubed. So faint trails are red, brighter ones warm through orange, and only the brightest go toward white — each channel switching on at a different intensity, from one tone-mapped number.

A warm amber ramp on black reads as bioluminescence rather than a heat map.

The alternative — a rainbow palette, which the same three-cosine trick from Gray-Scott would have given for free — would have made this look like a thermal image of something. The choice is about what the picture claims to be.

Where the borrowing shows

Three of these five are lifted from siblings, and each is labelled:

from	what
fluid	the $x/(1+x)$ tone curve; save_png verbatim
grayscale	the _at discipline, the flags-only view class, _pack
mandelbrot	no shader: the colour kernel writes BGRA the drawable takes directly

example folders are self-contained by this tree's own convention, so this is a copy, not an import

Copying rather than sharing keeps every example a folder that opens and runs on its own. It costs duplication and it is stated as the trade each time it is taken.

5. A frame, end to end

Three dispatches. This is the shortest frame in the collection.

Before the loop

```
ctx.enqueue_function(init_kern, px, py, pa, grid_dim=(AGENT_GRID), block_dim=(BLOCK))
ctx.enqueue_memset(trail_a, Float32(0.0))
ctx.enqueue_memset(trail_b, Float32(0.0))
ctx.synchronize()
```

Three agent arrays scattered by the init kernel, two trail buffers zeroed. The initial state is 300,000 agents at random positions on an **empty** map — so for the first few frames every sensor reads zero, every turn is a coin toss, and the agents wander. The structure appears as soon as enough trail exists for the sensors to have an opinion, which takes a couple of seconds and is worth watching.

Step 1: events, into flags

```
comptime CMD_CLICK = 1
comptime CMD_PAUSE = 2
comptime CMD_RESET = 4
comptime CMD_QUIT = 8
comptime CMD_PRESET = 16
comptime CMD_SAVE = 32
```

The same house rule as Fluid, Othello and Gray-Scott: the view's handlers set bits and nothing else, because a `DeviceContext` lives in a local that a Cocoa callback cannot reach. The pump owns the GPU.

Bits rather than booleans so two commands in one frame cannot lose each other.

And the same `let/var` discipline on the way out — read the flags out of the global before clearing, because `let` binds to a place in this dialect.

Step 2: the three dispatches

```
if not paused:
    ctx.enqueue_function(
        agent_kern, px, py, pa, cur, UInt32(frames),
        sensor_angle, turn_angle, sensor_dist,
        grid_dim=(AGENT_GRID), block_dim=(BLOCK),
    )
    ctx.enqueue_function(
        diffuse_kern, nxt, cur, decay, grid_dim=(GRID), block_dim=(BLOCK)
    )
    parity = 1 - parity
    cur = trail_a if parity == 0 else trail_b

ctx.enqueue_function(color_kern, dev, cur, grid_dim=(GRID), block_dim=(BLOCK))
ctx.synchronize()
```

Read the order carefully, because it matters.

The agent kernel writes into `cur` — the current trail map. Agents both sense and deposit into the same buffer, in the same dispatch. That is safe in a way the stencil kernels are not, because an agent reads three *scattered* points and writes one *unrelated* point; there is no neighbourhood being consumed. A deposit landing in a cell another agent is about to sense is not a correctness problem, it is the mechanism.

The diffuse kernel reads `cur` and writes `nxt` — separate buffers, because this one *is* a stencil and every cell needs its neighbours' old values.

Then the parity flips. So the fresh map becomes `cur` for the next frame, and the colour kernel reads it.

The parity trick, and how it differs from Gray-Scott's

Gray-Scott ping-pongs by **argument order** in an even loop:

```
for _step in range(SUBSTEPS // 2):  
    ctx.enqueue_function(gs_kern, u2, v2, u, v, ...)  
    ctx.enqueue_function(gs_kern, u, v, u2, v2, ...)
```

Twenty substeps, an even number, so the state always lands back in `u, v` and nothing has to track which buffer is current.

Physarum diffuses **once** per frame, so the buffer genuinely alternates and there is no even count to hide behind. It tracks the parity explicitly:

```
parity = 1 - parity  
cur = trail_a if parity == 0 else trail_b
```

Both are correct; the choice follows from the step count. An odd number of swaps per frame needs a variable, an even number does not — and Gray-Scott took the trouble to make its count even *because* it wanted to avoid this variable.

Note also that `paused` skips both simulation dispatches but **not** the colour kernel, so a paused frame is still drawn. The colour kernel is deliberately outside the `if`.

Step 3: the one blocking point

`ctx.synchronize()` after three enqueued dispatches. Everything before it was queued and is paid for once.

Then the copy to the host buffer, the optional PNG, and the present — all identical to Gray-Scott, including the comment:

```
# Save here, not later: `bgra` is exactly what is about to be  
# presented, so the file and the window cannot disagree.
```

Preset switching, mid-flight

```
if (pending & CMD_PRESET) != 0:
```

```
...
sensor_angle = preset_sensor_angle(preset)
turn_angle = preset_turn_angle(preset)
sensor_dist = preset_sensor_dist(preset)
decay = preset_decay(preset)
```

Four floats change and the next dispatch is a different regime — **the map is not cleared and the agents are not moved**. So an established web reorganises into a coil in front of you, which is far more informative than a fresh start: you see which structures survive the change and which dissolve.

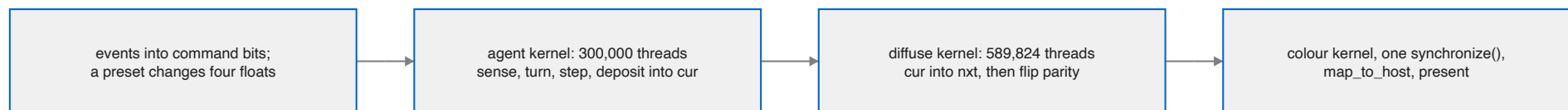
`r` does the other thing — reseeds the agents *and* clears both trail buffers, because agents scattered onto an old network would just re-follow it.

Verification: looking at the picture

The commit is explicit that a clean exit was not considered proof:

Verified the same way grayscott was: actual saved PNGs, not just a clean exit. The default preset produces the textbook thick branching vein network; switching to "coil" on the same agent count visibly thins the filaments into tighter curls — proof the parameters reach the kernel, not just a label changing.

The specific risk being tested for is worth naming: a preset switch that updated the window title and the local variables but failed to *pass* them to the dispatch would look completely correct from the outside. Every automated check would pass. Only two visibly different pictures rule it out.



6. What to understand

Eight things about this example are not obvious from watching it.

1. There is no network in the code

No graph, no edges, no nodes, no path-finding. An agent holds a position and a heading and nothing else — no memory of where it has been, and no way to perceive another agent.

No agent knows about any other agent, and no cell of the map knows it is part of a network.

The veins are what 300,000 copies of one rule do to a shared array. That is **stigmergy**, and it is the same mechanism that builds a termite mound.

2. Fading is what makes it work

Without decay, every path ever taken persists, the map saturates, all three sensors read alike, and the agents turn at random forever — a network that dissolves by remembering too much.

Decay makes a trail a claim that must be renewed. A route survives only while agents keep using it, which is exactly the dead-end withdrawal that made the real organism famous.

A trail is a rumour: it spreads to its neighbours and fades unless agents keep repeating it.

3. The blur and Gray-Scott's Laplacian are opposites

Both are nine-point stencils over the same neighbourhood.

	Physarum	Gray-Scott
weights sum to	one	zero
returns	a value — the local average	a rate of change
what it is	a blur	a Laplacian

Gray-Scott integrates its result into an existing field, so it needs a rate. Physarum replaces the value outright, so it wants the average.

4. The map wraps because the agents do

```
"""The trail map wraps, not clamps: an agent that walks off the right
edge reappears on the left, so the map it senses and deposits into
must wrap the same way or a trail would smear against a false wall
the agents themselves never see."""
```

Gray-Scott clamps, because its cells do not travel and a reflecting boundary is the physical choice. Physarum's agents travel, so a clamped map would give them a torus to walk on and a rectangle to sense on.

The two must agree. That is the transferable point: **topology is a property of the system, not of a function.**

5. Structure of arrays, not array of structures

Three separate `Float32` buffers for `x`, `y` and `angle`. Thread `n` reads `px[n]` and thread `n+1` reads `px[n+1]`, so adjacent threads touch adjacent addresses and the loads coalesce. An array of `{x, y, a}` structs would stride each thread past two floats it does not want.

Slightly worse to read, meaningfully better to run.

6. The deposit races on purpose

```
trail[unsafe_offset=cidx] = trail[unsafe_offset=cidx] + DEPOSIT
```

Read-modify-write from 300,000 threads, not atomic. Two agents on the same cell in the same step can lose an increment — and the file says so, and says why it does not matter:

An atomic add would buy correctness this demo has no way to show.

The lost value is 5.0 on one of 589,824 cells for one frame, before being averaged with eight neighbours and decayed.

Compare fernwind, which does the same scattered accumulation and *does* use `Atomic.fetch_add` — because there the count is a divisor for a mean colour, so lost increments would bias the result rather than dither it.

Same operation, opposite decisions, both reasoned from what the number is used for afterwards. *"Is this racy?"* is the wrong question; *"what would a lost increment change?"* is the right one.

7. Hash the index instead of carrying an RNG

```
var r = _rand01(UInt32(idx) * UInt32(747796405) + frame)
```

No per-agent generator state to allocate, seed or advance — and no setup kernel. Index and frame in, a number out.

The `frame` term is load-bearing: without it, an agent that hit a sensor tie would break it the same way every frame forever.

Note `_rand01` takes 24 bits, which is what a `Float32` mantissa holds exactly.

8. The mouse has no attraction code

Nothing forces an agent toward it — the network only bends here because sensing a bright patch and turning toward it is all an agent ever does anyway; this just gives it something bright to find.

There is no force, no target, no steering behaviour. The mouse writes into the map the agents were already reading. Interaction is free because it reuses the mechanism that was already the entire model.

Things that will bite: the model

If you change...	...this happens
decay to 1.0	trails never fade; the map saturates and the network dissolves into noise
the blur weights to sum to something else	the map gains or loses trail everywhere, on its own
_at to clamp instead of wrap	trails pile against a wall the agents cannot see
the tie-break to leave the heading alone	agents walk straight until they chance on a trail; the map stays blank far longer
the step to under one pixel	agents redeposit into the same cell; trails come out dotted

Things that will bite: the machinery

If you change...	...this happens
the three agent arrays into one struct array	the loads stop coalescing
diffuse to write in place	threads read a mix of old and new neighbours; nondeterministic
the frame term out of the tie-break hash	an agent breaks every tie the same way, forever
the three init multipliers to one seed	every agent starts on a diagonal with a correlated heading
the parity flip	the colour kernel draws the pre-diffusion map, every frame
_rand01 to take all 32 bits	the divisor is not exactly representable; the low bits are lost

Controls

drag	lay a strong trail — the network bends to it
1–4	web, coil, sparse, dense — live, map kept
r	reseed agents, clear both trail buffers
space / s / q	pause · save a PNG · quit
PHYSARUM_FRAMES=N	render N frames unfocused, then exit

And beside the others

example	what couples the work	dispatches per frame
mandelbrot	nothing	1
physarum	a shared map, written racily	3
grayscott	one cell of diffusion per substep	21
fluid	a global constraint needing a solver	~35

Physarum is cheap because its coupling is weak — agents affect each other only through a map that is blurred and decayed anyway. Fluid is expensive because its coupling is exact. Reading the four together is the clearest picture in the collection of what coupling actually costs.